Free Grammars – II - new

Translated and adapted by L. Breveglieri

PARENTHESES LANGUAGE: artificial languages frequently contain nested structures, i.e., parentheses, where an element pair starts and ends some substructure (like for instance a substring), and where the pair may in turn contain a nested pair, and so on recursively down to arbitrary depth

Pascal:	begin ... end
C:	{ … }
XML:	< title > … < /title >
LaTeX:	\begin{equation} … \end{equation}

Do not consider the specific way the marker symbols are encoded

The paradigm of parenthesis languages is known as the *Dyck Language*

Alphabet $\Sigma = \{')','(',']','['\}$

Phrases $()[[()[]]()]$

Parenthesis phrases can be equivalently defined by means of *cancellation rules*: repeatedly remove from the string any factor that consists of a pair of adjacent open and closed parentheses, as long as it is possible.
The original string is valid if and only if the final result is the *empty* string

$$[\ ] \Rightarrow \varepsilon \qquad (\ ) \Rightarrow \varepsilon$$

DYCK LANGUAGE: open parentheses *a, b, ...,* closed parentheses *a', b', ...*

$$\Sigma = \{a, a', b, b'\}$$

$$S \to aSa'S \mid bSb'S \mid \varepsilon$$

$$\underbrace{\underbrace{a\,a\,\underbrace{aa'}\,a'}\,\underbrace{a\,a\,\underbrace{aa'}\,a'}\,a'\,a'}$$

LINEAR BUT NON-REGULAR LANGUAGE

$$L_1 = \{a^n c^n \mid n \geq 1\} = \{ac, aacc, \ldots\}$$

$$S \to aSc \mid ac$$

L_1 is a subset of the Dyck language.
It does not admit more than one parenthesis nest

REGULAR COMPOSITION OF FREE LANGUAGES

The basic regular operations (union, concatenation and star), applied to free languages, still yield free languages

The family of free languaegs is closed with respect to the language operations *union*, *concatenation* and *star*

$$G_1 = \left(\Sigma_1, V_{N_1}, P_1, S_1\right) \text{ e } G_2 = \left(\Sigma_2, V_{N_2}, P_2, S_2\right)$$

$$V_{N_1} \bigcap V_{N_2} = \varnothing \qquad S \notin (V_{N_1} \bigcup V_{N_2})$$

UNION

$$G = (\Sigma_1 \bigcup \Sigma_2, \{S\} \bigcup V_{N_1} \bigcup V_{N_2}, \{S \to S_1 \mid S_2\} \bigcup P_1 \bigcup P_2, S)$$

CONCATENATION

$$G = (\Sigma_1 \bigcup \Sigma_2, \{S\} \bigcup V_{N_1} \bigcup V_{N_2}, \{S \to S_1 S_2\} \bigcup P_1 \bigcup P_2, S)$$

STAR: G of $(L_1)^*$ is obtained by adding the rules $S \to S\ S_1 \mid \varepsilon$ to G_1

CROSS: G of $(L1)^+$ is obtained by adding the rules $S \to S\ S_1 \mid S_1$ to G_1

EXAMPLE: union of languages

$$L = \left\{ a^i b^i c^* \mid i \geq 0 \right\} \bigcup \left\{ a^* b^i c^i \mid i \geq 0 \right\} = L_1 \bigcup L_2$$

G_1

$S_1 \rightarrow XC$

$X \rightarrow aXb \mid \varepsilon$

$C \rightarrow cC \mid \varepsilon$

G_2

$S_2 \rightarrow AY$

$Y \rightarrow bYc \mid \varepsilon$

$A \rightarrow aA \mid \varepsilon$

$$\left\{ S \rightarrow S_1 \mid S'' \right\} \bigcup P_1 \bigcup P''$$

G''

$S'' \rightarrow AX$

$X \rightarrow bXc \mid \varepsilon$

$A \rightarrow aA \mid \varepsilon$

CAUTION: if the hypothesis that the two non-terminal alphabets are disjoint does not hold, then the construction above yields a grammar that generates a superset of the real union language. For example, replacing G_2 with G'' would allow us to generate the invalid phrase shown aside

$abcbc$

The family LIB of free languages is closed with respect to STRING MIRRORING

Given the grammar G of the language, the grammar G_R that generates the mirror image of the language is obtained from G by mirroring the right member of every rule of G

REG and LIB are both closed with respect to union, concatenation and star

but later it will be proved that

they do not behave in the same way as for complement and intersection

AMBIGUITY: semantic versus syntactic
“Vedo un uomo in giardino con il cannocchiale”
(do I watch a man who has a spyglass or do I use a spyglass to watch a man ?)
“La pesca è bella” (the peach is fine or the sport of fishing is fine ?)
“half baked chicken” (a chicken that is half baked or a half chicken that is baked ?)

Natural languages are largely ambiguous and this phenomenon is unavoidable, as they aim at describing everything, but with only a finite lexicon (dictionary). Instead, in the artifical languages ambiguity is an undesirable phenomenon (think of a program, how could we tolerate ambiguity ?)
Therefore ambiguity must be eliminated (if possible) or kept under control

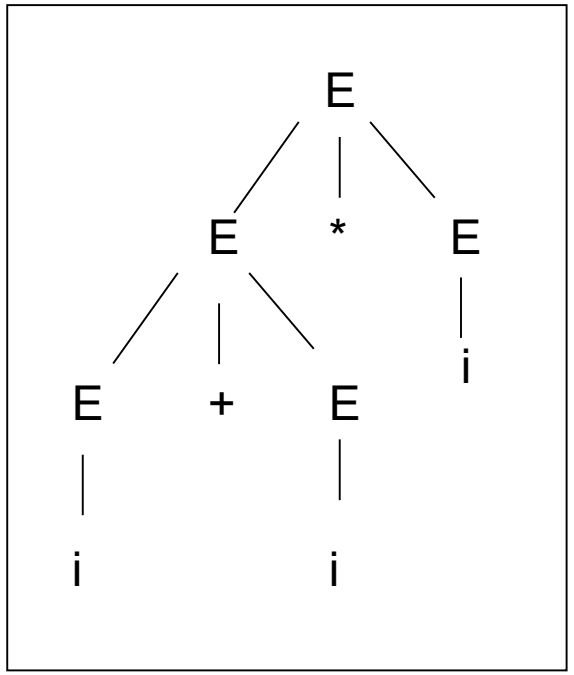
Consider SYNTACTIC AMBIGUITY. A phrase *x* of the language defined by the grammar G is said to be ambiguous if it admits two or more different syntactic trees (or, equivalently, two different derivations). A grammar G is said to be ambiguous if it generates (at least) an ambiguous string

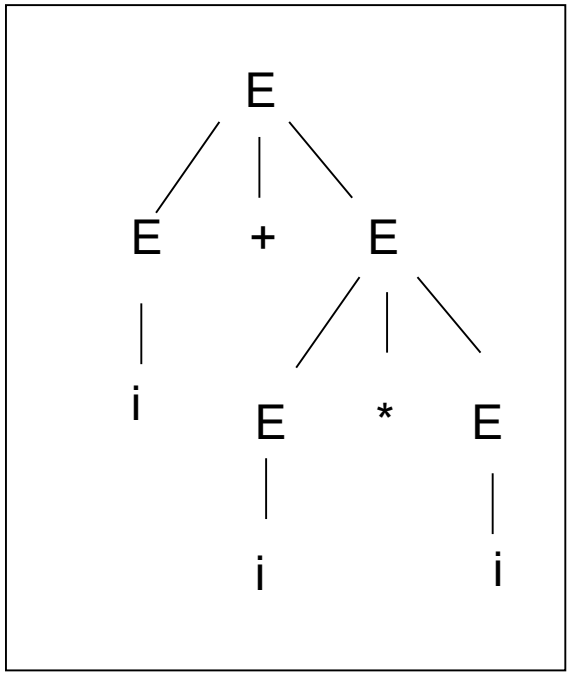
EXAMPLE – grammar G’ of the arithmetic expressions

$$E \rightarrow E + E \mid E * E \mid (E) \mid i$$

$$E \Rightarrow E * E \Rightarrow E + E * E \Rightarrow i + E * E \Rightarrow i + i * E \Rightarrow i + i * i$$

$$E \Rightarrow E + E \Rightarrow i + E \Rightarrow i + E * E \Rightarrow i + i * E \Rightarrow i + i * i$$





The phrase *i* + *i* * *i* is ambiguous. Therefore grammar G' is itself ambiguous

In fact, grammar G' is not compliant with the standard convention
that multiplication must precede addition

Previously, an unambiguous grammar G that generates the arithmetic expressions has been shown. G' is equivalent to G (that is, L(G) = L(G')), but is smaller than G. However, grammar G' is ambiguous. This is often the case: simplifying the grammar frequently ends up with having an ambiguous behaviour, in general

The AMBIGUITY DEGREE of a phrase *x* in the language L(G) is the number of different syntax trees that *x* admits. The ambiguity degree of a string may be infinite (only if there are nullable non-terminal symbols)
The AMBIGUITY DEGREE of a grammar G is the maximum ambiguity degree of the strings generated by G. While every string of L(G) may have a finite ambiguity degree, the ambiguity degree of G may still turn out to be unbounded

RELEVANT PROBLEM: decide whether a grammar G is ambiguous or not.
UNFORTUNATELY, THIS PROBLEM IS UNDECIDABLE: there does not exist any algorithm to decide whether a given grammar G is ambiguous or not.
This can be done for some grammars, but in general not for every grammar

By using the methods of theoretical information science, it is possible to prove that any potential decision algorithm should examine increasingly long derivations, which is unfeasible and hence leads to undecidability. However, the inexistence of a general algorithm does not preclude to decide for a specific grammar, by resorting to *ad hoc* methods depending on the structure of the grammar. From a practical point of view, examining all the derivations of G up to a given length is often sufficient to get convinced that the grammar is unambiguous, though it is not a rigorous proof

AS AMBIGUITY MAY BE DIFFICULT TO IDENTIFY A POSTERIORI
IT IS ADVISABLE TO AVOID IT BY CONSTRUCTION (A FORTIORI)

EXAMPLE: resume the previous example

The phrase $i + i + i$ has ambiguity degree equal to 2

The phrase $i + i * \mathrm{i} + i$ has ambiguity degree equal to 5 (see below)

$$\underbrace{\underbrace{i+i}*\underbrace{i+i}}, \quad \underbrace{i+\underbrace{i*i}+i}, \quad \underbrace{i+\underbrace{i*i}+i}, \quad \underbrace{\mathrm{i}+\underbrace{\mathrm{i}*\underbrace{\mathrm{i}+\mathrm{i}}}}, \quad \underbrace{\underbrace{i+i}*i+i}$$

Typically, as the derived phrase gets longer, its ambiguity degree grows as well

AMBIGUITY OF TWO-SIDED RECURSION: a non-terminal symbol that is recursive both on the left and on the right (two-sided recursion) always allows two or more derivations, and thus gives rise to an ambiguous behaviour (see below)

$$A \overset{+}{\Rightarrow} A \ldots \quad A \overset{+}{\Rightarrow} \ldots A$$

EXAMPLE 1: grammar G_1 generates the phrase *i* + *i* + *i* in two different ways (by means of two different leftmost derivations)

ambiguous grammar

$$G_1 : \quad E \rightarrow E + E \mid i$$

notice that L is regular

$$L(G_1) = i(+i)^*$$

non-ambiguous version

$$E \rightarrow i + E \mid i$$

$$E \rightarrow E + i \mid i$$

EXAMPLE 2: ambiguity that originates form left and right recursion, separately

ambiguous grammar

$$G_2: \quad A \to aA \mid Ab \mid c$$

ambiguity can be removed by separately generating the two lists

$$L(G_2) = a^* c b^*$$

$$S \to AcB$$

$$A \to aA \mid \varepsilon$$

$$B \to bB \mid \varepsilon$$

or one may choose to orderly generate the two lists (first *a* then *b*, or viceversa)

$$L(G_2) = a^* c b^*$$

$$S \to aS \mid X$$

$$X \to Xb \mid c$$

AMBIGUITY OF UNION (maybe the simplest to understand)

If two languages $L_1(G_1)$ and $L_2(G_2)$ share some phrases (i.e., their intersection is not empty), the grammar G of the union language, constructed as shown before, is surely ambiguous

CAUTION: one needs to assume that the two non-terminal sets are disjoint, otherwise, as seen before, the union grammar would generate a superlanguage that strictly contains both languages

A phrase *x* that belongs to the union of L_1 and L_2, and admits two different derivations, the first only using rules of G_1 and the second only using rules of G_2, is ambiguous for the grammar G, as G contains both sets of rules. Only the phrases that belong to $L_1 \setminus L_2$ or to $L_2 \setminus L_1$, if any, would be generated in a non-ambiguous way

EXAMPLE 3

$$\boxed{S \to bS \mid cS \mid \varepsilon}$$

non-ambiguous version

ambiguous grammar

$$G: \quad S \to bS \mid cS \mid D \quad D \to bD \mid cD \mid \varepsilon$$

$$L(G) = L_S(G) \bigcup L_D(G)$$

$$L_S(G) = \{b, c\}^* = L_D(G)$$

$$S \overset{+}{\Rightarrow} bbcD \Rightarrow bbc \quad S \Rightarrow D \overset{+}{\Rightarrow} bbcD \Rightarrow bbc$$

EXAMPLE 4

$$\boxed{\begin{aligned} S &\to B \mid D \mid \varepsilon \\ B &\to bBc \mid bc \\ D &\to dDe \mid de \end{aligned}}$$

non-ambiguous version

ambiguous grammar

$$S \to B \mid D \quad B \to bBc \mid \varepsilon$$

$$D \to dDe \mid \varepsilon$$

B generates $b^n c^n, n \geq 0$

D generates $d^n e^n, n \geq 0$

ε is the only ambiguous phrase

LANGUAGES THAT ARE INHERENTLY AMBIGUOUS

A language is said to be INHERENTLY AMBIGUOUS if every grammar that generates it is necessarily ambiguous, i.e., if all the equivalent grammars of the language happen to be ambiguous

EXAMPLE: here is a classical inherently ambiguous language

$$\boxed{L = \left\{ a^i b^j c^k \mid (i, j, k \geq 0) \wedge ((i = j) \vee (j = k)) \right\}}$$

$$\boxed{L = \left\{ a^i b^i c^* \mid i \geq 0 \right\} \bigcup \left\{ a^* b^i c^i \mid i \geq 0 \right\} = L_1 \bigcup L_2}$$

ambiguous union

Whatever grammar G is designed to generate L, the phrases ε, abc, $a^2b^2c^2$, ...
ALWAYS HAPPEN TO BE DERIVABLE IN TWO OR MORE WAYS.
This behaviour is caused by the very nature of the language L
and does not depend on the specific grammar G that generates L

In fact, such phrases can be generated by G_1 to check that $|x|_a = |x|_b$

$$a \ldots a \underbrace{\underbrace{a b}_{} b \ldots b}_{} bcc \ldots c$$

generated by G_1

or can be generated by G_2 to check that $|x|_b = |x|_c$

$$a \ldots aa \underbrace{\underbrace{b \ldots b \underbrace{bc}_{} c}_{} \ldots c}_{}$$

generated by G_2

But clearly, since both sub-grammars can generate such phrases, ambiguity arises. In whatever way the two sub-grammars are modified, both checks are unavoidable, as they belong to the structure of the language, and therefore ambiguity is unavoidable as well

AMBIGUITY OF CONCATENATION

Concatenating two languages may give rise to ambiguity if there exists a suffix of a phrase of the former language that is a prefix of a phrase of the latter language

$$\boxed{G = (\Sigma_1 \cup \Sigma_2, \{S\} \cup V_{N_1} \cup V_{N_2}, \{S \to S_1 S_2\} \cup P_1 \cup P_2, S)}$$

Assume that G_1 and G_2 are not ambiguous, then G is ambiguous if there exist two phrases x' $\in L_1$ and x'' $\in L_2$, and a non-empty string *v* such that

$$x' = u'v \wedge u' \in L_1 \quad x'' = vz'' \wedge z'' \in L_2$$

$u'vz'' \in L_1.L_2$ and is ambiguous

$$S \Rightarrow S_1 S_2 \overset{+}{\Rightarrow} u' S_2 \overset{+}{\Rightarrow} u'vz''$$

$$S \Rightarrow S_1 S_2 \overset{+}{\Rightarrow} u'vS_2 \overset{+}{\Rightarrow} u'vz''$$

EXAMPLE 1 – concatenation of Dyck languages

$$\Sigma_1 = \{a, a', b, b'\} \quad \Sigma_2 = \{b, b', c, c'\}$$

$$aa'bb'cc' \in L = L_1 L_2$$

$$G(L): \quad S \to S_1 S_2$$

$$S_1 \to aS_1a'S_1 \mid bS_1b'S_1 \mid \varepsilon$$

$$S_2 \to bS_2b'S_2 \mid cS_2c'S_2 \mid \varepsilon$$

$$\overbrace{aa'bb'}^{S_1}\overbrace{cc'}^{S_2} \qquad \overbrace{aa'}^{S_1}\overbrace{bb'cc'}^{S_2}$$

In order to eliminate ambiguity, it is necessary to exclude the case when a suffix moves from the former language to the latter one. A solution consists of inserting a separator between the two languages. Such a separator may not belong to the alphabets of the two languages. The concatenation language $L_1 \# L_2$ is generated by the additional axiomatic rule $S \to S_1 \# S_2$

OTHER AMBIGUOUS SITUATIONS – regexps may be themselves ambiguous

ambiguous grammar

$$S \to DcD \quad D \to bD \mid cD \mid \varepsilon$$

$$\{b,c\}^* \, c \, \{b,c\}^*$$

EXAMPLE 1

Every phase that contains two or more *c* is ambiguous. To remove ambiguity, one can impose that the mandatory *c* is the leftmost one

unambiguous version

$$S \to BcD \quad D \to bD \mid cD \mid \varepsilon \quad B \to bB \mid \varepsilon$$

EXAMPLE 2 – Fixing the order of application of the rules – The rule to generate two *b* can be applied before or after the rule that generates one *b*

$$S \to bSc \mid bbSc \mid \varepsilon$$

$$S \Rightarrow bbSc \Rightarrow bbbScc \Rightarrow bbbcc$$

$$S \Rightarrow bSc \Rightarrow bbbScc \Rightarrow bbbcc$$

ambiguous grammar

$$S \to bSc \mid D \quad D \to bbDc \mid \varepsilon$$

unambiguous version

GRAMMAR NORMAL FORM

Normal forms impose restrictions to the rules, but without reducing the family of generated languages. Such forms are useful for instance to prove theorems, rather than to design a grammar for a specific language

There are some transformations to put a general grammar into an equivalent normal form. Such transformations are helpful to design syntactic analyzers

GRAMMAR to start from

$$\boxed{G = \{\Sigma, V, P, S\}}$$

NON-TERMINAL EXPANSION

Expanding a non-terminal does not change the generative power of the grammar

$$A \rightarrow \alpha B \gamma \qquad B \rightarrow \beta_1 \mid \beta_2 \mid \ldots \mid \beta_n$$

$$A \rightarrow \alpha\beta_1\gamma \mid \alpha\beta_2\gamma \mid \ldots \mid \alpha\beta_n\gamma$$

$$A \Rightarrow \alpha B \gamma \;\; \Rightarrow \alpha\beta_i\gamma$$

$$A \Rightarrow \alpha\beta_i\gamma$$

REMOVING THE AXIOM FROM THE RIGHT MEMBER OF THE RULES

It is always possible to restrict the right member of a rule so as it is a string over the alphabet ($\Sigma \cup (V - \{ S \})$), i.e., it does not contain the axiom S

It suffices to introduce the new axiom S_0 and the new rule $S_0 \rightarrow S$

NULLABLE NON-TERMINAL SYMBOL and NON-NULLABLE NORMAL FORM

A non-terminal A is said to be *nullable* if and only if there exists a derivation as aside

$$\boxed{A \overset{+}{\Longrightarrow} \varepsilon}$$

CAUTION: this does not exclude that A generates also a string different from ε

$Null \subseteq V$ is the set of the nullable non-terminal symbols

Computation of the set of the nullable non-terminal symbols

$A \in Null$ if $A \rightarrow \varepsilon \in P$

$A \in Null$ if $(A \rightarrow A_1 A_2 \ldots A_n \in P$ with $A_i \in V \setminus \{A\}) \wedge \forall A_i (A_i \in Null)$

COPY RULES (cathegorization) AND HOW TO ELIMINATE THEM

Copy rule (or cathegorization rule): in the example, the syntactic class B is included in the syntatctic class A. By removing the copy rules, one obtains an equivalent grammar with syntactic trees that are less deep

Copy (A) $\subseteq$ V: the set of the non-terminal symbols into which A can be copied, possibly in a transitive way

$$A \rightarrow B \quad B \in V$$

iterative_phrase $\rightarrow$ *while_phrase* | *for_phrase* | *repeat_phrase*

$$Copy(A) = \left\{ B \in V \mid \text{there exists the derivation } A \overset{*}{\Rightarrow} B \right\}$$

Copy rules are not totally useless, as sometimes they allow us to share some parts of the grammar. For this reason copy rules may be used in the grammars of technical languages, like for instance in the programming languages

EXAMPLE – grammar of the arithmetic expressions without copy rules

standard non-ambiguous grammar
but with copy rules

$$E \to E + T \mid T \quad T \to T \times C \mid C$$

$$C \to 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

$$Copy\ (E) = \{E, T, C\} \quad Copy\ (T) = \{T, C\}$$

$$Copy\ (C) = \{C\}$$

equivalent grammar without copy rules

$$E \to E + T \mid T \times C \mid 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

$$T \to T \times C \mid 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

$$C \to 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

CHOMSKY NORMAL FORM: the production rules are only of two types

1. binary rule (of the homogeneous type) $\boxed{A \to BC \quad \text{where} \quad B, C \in V}$

2. terminal rule (with one-letter right side) $\boxed{A \to \alpha \quad \text{where} \quad \alpha \in \Sigma}$

If the language contains the empty string, add the rule $\boxed{S \to \varepsilon}$

Structure of the syntax tree: every internal node has arity 2,
and every node that is parent of a leaf has arity 1

HOW TO TRANSFORM LEFT RECURSION INTO RIGHT RECURSION

Construction of the NON-LEFT RECURSIVE FORM (indispensable for designing left recursive descent syntactic analyzers)

TRANSFORMATION OF IMMEDIATE LEFT RECURSION

$$A \rightarrow A\beta_1 \mid A\beta_2 \mid \ldots \mid A\beta_h$$

where no β_i is empty

$$A \rightarrow \gamma_1 \mid \gamma_2 \mid \ldots \mid \gamma_k$$

$$A \rightarrow \gamma_1 A' \mid \gamma_2 A' \mid \ldots \mid \gamma_k A' \mid \gamma_1 \mid \gamma_2 \mid \ldots \mid \gamma_k$$

$$A' \rightarrow \beta_1 A' \mid \beta_2 A' \mid \ldots \mid \beta_h A' \mid \beta_1 \mid \beta_2 \mid \ldots \mid \beta_h$$

$$A \Rightarrow A\beta_2 \Rightarrow A\beta_3\beta_2 \Rightarrow \gamma_1\beta_3\beta_2$$

$$A \Rightarrow \gamma_1 A' \Rightarrow \gamma_1\beta_3 A' \Rightarrow \gamma_1\beta_3\beta_2$$

EXAMPLE – how to shift a left immediate recursion to the right

$E \to E + T \mid T \qquad T \to T * F \mid F \qquad F \to (E) \mid i$

$E \qquad T$ are immediately recursive on the left

$E \to TE' \mid T \qquad E' \to +TE' \mid +T$

$T \to FT' \mid F \qquad T' \to *FT' \mid *F \qquad F \to (E) \mid i$

simple transformation but does not always work

$E \to T + E \mid T \qquad T \to F * T \mid F \qquad F \to (E) \mid i$

REAL TIME NORMAL FORM (and GREIBACH NORMAL FORM)

In the REAL TIME NORMAL FORM every rule begins with a terminal symbol

$$\boxed{A \rightarrow a\alpha \quad \text{where} \quad a \in \Sigma, \alpha \in \left\{\Sigma \bigcup V\right\}^*}$$

The GREIBACH NORMAL FORM is a special case of the above

Every rule begins with a terminal symbol, followed by zero or more non-terminal symbols

$$\boxed{A \rightarrow a\alpha \quad \text{where} \quad a \in \Sigma, \alpha \in V^*}$$

"REAL TIME" - this term is justified as a property of the syntax analysis algorithm, which reads and consumes exactly one terminal symbol at every step. Therefore, the number of steps needed to complete the analysis is equal to the length of the string to analyze

UNI-LINEAR GRAMMAR (or GRAMMAR of TYPE 3)

RIGHT UNI-LINEAR RULE (or simply RIGHT LINEAR)

$$\boxed{A \to uB \quad \text{where} \quad u \in \Sigma^*, B \in (V \bigcup \varepsilon)}$$

LEFT UNI-LINEAR RULE (or simply LEFT LINEAR)

$$\boxed{A \to Bv \quad \text{where} \quad v \in \Sigma^*, B \in (V \bigcup \varepsilon)}$$

The corresponding syntax trees grow in a heavily unbalanced way

If a left or right uni-linear grammar contains a recursive derivation, then such a derivation is recursive on the left or right, respectively

THE ROLE OF SELF-INCLUSIVE DERIVATIONS

$$\boxed{A \overset{+}{\Rightarrow} uAv \quad u \neq \varepsilon \wedge v \neq \varepsilon}$$

A self-inclusive derivation may not occur in the unilinear grammar of regular languages. This does not happen in the free grammars that generate free non-regular languages (palindromes, Dyck, etc)

The absence of self-inclusive derivations allows to solve the set-theoretic equations associated with unilinear grammars, and gives rise to the simple structure of the languages that such grammars generate

A GRAMMR THAT DOES NOT ADMIT ANY SELF-INCLUSIVE DERIVATION ALWAYS GENERATES A REGULAR LANGUAGE

THE CLOSURE PROPERTIES OF THE REG AND LIB FAMILIES

Here the closure properties of the REG and LIB families of languages are examined, with respect to the most common language operators

Let L and R be two languages belonging to LIB and REG, respectively

$R^R \in REG$	$L^R \in LIB$
$R^* \in REG$	$L^* \in LIB$
$\neg R \in REG$	$\neg L \notin LIB$
$R_1 \oplus R_2 \in REG$	$L_1 \oplus L_2 \in LIB$
$R_1 \bigcap R_2 \in REG$	$L_1 \bigcap L_2 \notin LIB$
	$L \bigcap R \in LIB$

denotes both union and concatenation

denotes both union and concatenation

FREE GRAMMARS EXTENDED BY MEANS OF REGULAR EXPRESSIONS

REGULAR EXPRESSIONS ARE MORE READABLE AND PERSPICUOUS than free grammars, thanks to the iteration operators (star and cross) and to the choice operator (union)

To make free grammars more readable as well, it is often permitted to use regexps also in the right sides of the rules of the grammar. One thus obtains the so-called EXTENDED FREE GRAMMARS (Extended BNF or EBNF). The grammars of the technical languages (programming languages, description languages, etc) are usually presented in an extended form

The EBNF rules can be represented as syntax diagrams. A syntax diagram can be conceived as the flow graph of a syntax analysis algorithm

The family LIB of free languages is closed with respect to regular operators (union, star, concatenation); therefore the extended free grammars have the same generating power as the non-extended ones

EXAMPLE: list of declarations of variable names

$$\textit{character}\ \ \text{testo1, testo2;}\ \textit{real}\ \ \text{temp, result;}$$

$$\textit{integer}\ \ \text{alfa, beta2, gamma;}$$

$\Sigma = \{c, i, r, v,\ ',',\ ';'\}$ where *v* is a variable name

$$\left(\left(c \mid i \mid r\right) v(,v)^{*};\right)^{+}$$

$$S \to SE \mid E \quad E \to AF; \quad A \to c \mid i \mid r \quad F \to v, F \mid v$$

The grammar is longer than the regexp, and makes less evident the existence of a hierarcy of two lists. Moreover, the names A, E and F are arbitrary

A FREE EXTENDED GRAMMAR (EBNF) G = { V, Σ, P, S } contains exactly $|V|$ rules, each of the form A → η, where η is a regexp over the alphabet V U Σ. For better clarity it is allowed to use derived regular operators, like for instance cross, optionality, repetition, etc

EXAMPLE: a simplified Algol-like language

$$B \rightarrow b\left[D\right]Ie$$

$$D \rightarrow ((c \mid i \mid r)v(,v)^{*};)^{+}$$

$$I \rightarrow F(;F)^{*}$$

$$F \rightarrow a \mid B$$

DERIVATION AND SYNTAX TREE IN THE EXTENDED GRAMMARS

The right side of an extended rule is a regexp that defines an infinite set of strings

Such strings could be imagined as the right sides of a grammar G', equivalent to G, except that it contains infinitely many rules

$$A \rightarrow (aB)^{+}$$

$$A \rightarrow aB \mid aBaB \mid \ldots.$$

DERIVATION RELATION IN AN EXTENDED GRAMMAR

given the strings $\eta_1 \quad \eta_2 \in (\Sigma \bigcup V)^{*}$

say that η_2 *derives* (immediately) from η_1

$\eta_1 \Rightarrow \eta_2$ if the two strings are factored as follows

$\eta_1 = \alpha A \gamma, \quad \eta_2 = \alpha \vartheta \gamma$ and there is a rule

$A \rightarrow e: \quad e \overset{*}{\Rightarrow} \vartheta$

note that $\eta_1 \quad \eta_2 \; (\quad \alpha A \gamma \quad \alpha \vartheta \gamma)$ do not contain

either regular operators or parentheses, and *e* is a regexp

Similarly, one can define multiple step derivations and the generated language

EXAMPLE: extended derivation for arithmetic expressions

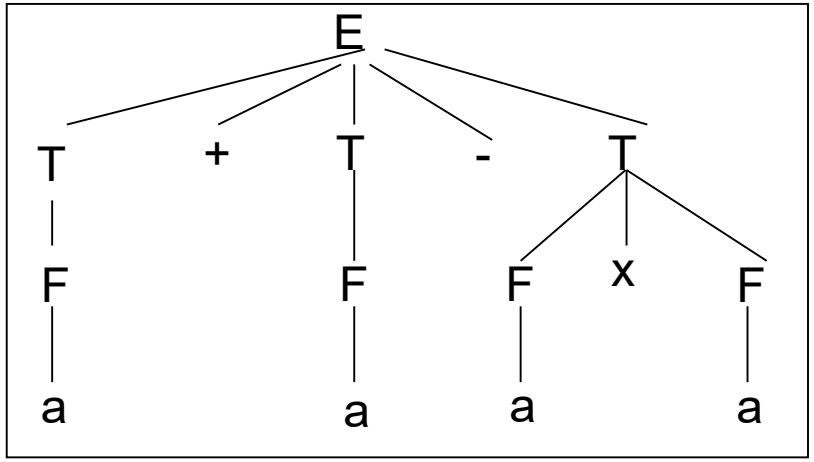
The extended grammar G shown below generates the arithmetic expressions with the four standard operators (in infix notation), round brackets and the symbol *a* to represent generic variables or constants

$$E \Rightarrow [+|-]T((+|-)T)^* \quad T \rightarrow F((\times|/)F)^* \quad F \rightarrow (a|'('E')')$$

leftmost derivation

$$E \Rightarrow T+T-T \Rightarrow F+T-T \Rightarrow a+T-T \Rightarrow a+F-T \Rightarrow$$

$$\Rightarrow a+a-T \Rightarrow a+a-F\times F \Rightarrow a+a-a\times F \Rightarrow a+a-a\times a$$



In EBNF, the arity of a tree node may be unlimited, in general

However, as the tree gets larger the depth gets lower

AMBIGUITY IN EXTENDED GRAMMARS

A non-extended but ambiguous grammar is ambiguous also when conceived as extended (as BNF is a special case of EBNF)

However, the presence of regexps in the rules may give rise to specific ambiguity forms

$a^*b \mid ab^*$ numbered as $a_1^*b_2 \mid a_3b_4^*$

is ambiguous because ab is

derivable as a_1b_2 or as a_3b_4

$S \rightarrow a^*b \mid ab^*$ is ambiguous as well

MORE GENERAL GRAMMARS – CHOMSKY CLASSIFICATION

Grammar	*Rule Type*	*Lang. Family*	*Recognizer Device*
Type 0 recursively denumerable	$\beta \to \alpha$ where $\alpha, \beta \in (\Sigma \cup V)^+$; $\beta \neq \varepsilon$	Recursively denumerable languages	Turing machine
Type 1 context-dependent	$\beta \to \alpha$ where $\alpha, \beta \in (\Sigma \cup V)^+$; $\lvert \beta \rvert \leq \lvert \alpha \rvert$	Context-dependent languages	Turing machine with a memory tape of length equal to that of the string to be recognized

CHOMSKY CLASSIFICATION (continued)

Grammar	*Rule Type*	*Lang. Family*	*Recognizer Device*
Type 2 context- free or BNF form	$A \rightarrow \alpha$ where A is nonterm. $\alpha \in (\Sigma \bigcup V)^*$	Contex-free languages or BNF languages or algebraic languages	Pushdown automaton (non-deterministic)
Type 3 right or left uni-linear	R-lin: $: A \rightarrow uB$ L-lin: $: A \rightarrow Bu$ where A is nonterm. $u \in \Sigma^*$ $B \in (V \bigcup \varepsilon)$	Regular or rational languages	Finite state automaton or sequential machine